PROJECT BASED LEARNING | REVIEW 2

# Hotel Reservation & Guest Services Management System

Normalization, 3NF Logical Design and Database Implementation

| Subject | Phase |
|---|---|
| Database Management Systems | Review 2 of 3 |

## 1. Recap and Objective of Review 2

Review 1 defined the problem, scope, users, key rules, workflow and the conceptual design (ER diagram with 11 entities and an initial relational schema). Review 2 converts that conceptual design into a **normalized, constraint-protected and working database**.

> **Objective of Review 2**
> To normalize the initial relational schema up to Third Normal Form (3NF), implement it as SQL DDL with integrity constraints, load realistic sample data, and verify the design through a working database-connected prototype.

| Review 1 (Completed) | Review 2 (This Document) | Review 3 (Next) |
|---|---|---|
| Problem and scope | Functional dependencies | SQL queries and reports |
| Users and key rules | Normalization 1NF to 3NF | Full application workflow |
| ER diagram | DDL with constraints | Testing: 21 test cases |
| Initial relational schema | Sample data (seed) | Results and future scope |

## 2. Implementation at a Glance

The design has been implemented as a working prototype so that every normalization and constraint decision can be demonstrated on live data.

| Item | Implementation |
|---|---|
| Database engine | SQLite (file: database/hotel.db) with PRAGMA foreign_keys = ON |
| Application | Python Flask web application (app.py) with Jinja2 templates and CSS |
| Tables | 11 (room_type, room, rate_plan, guest, reservation, stay, service_request, housekeeping_task, service_charge, invoice, payment) |
| Attributes | 66 columns across the 11 tables |
| Constraints | 11 primary keys, 11 foreign keys, UNIQUE keys, CHECK constraints, DEFAULT values |
| Indexes | 3 performance indexes and 1 partial unique index |

| Item | Implementation |
|---|---|
| Project files | schema.sql (DDL), seed.sql (sample data), queries.sql (12 queries), docs/ (FDs, normalization, ERD source) |

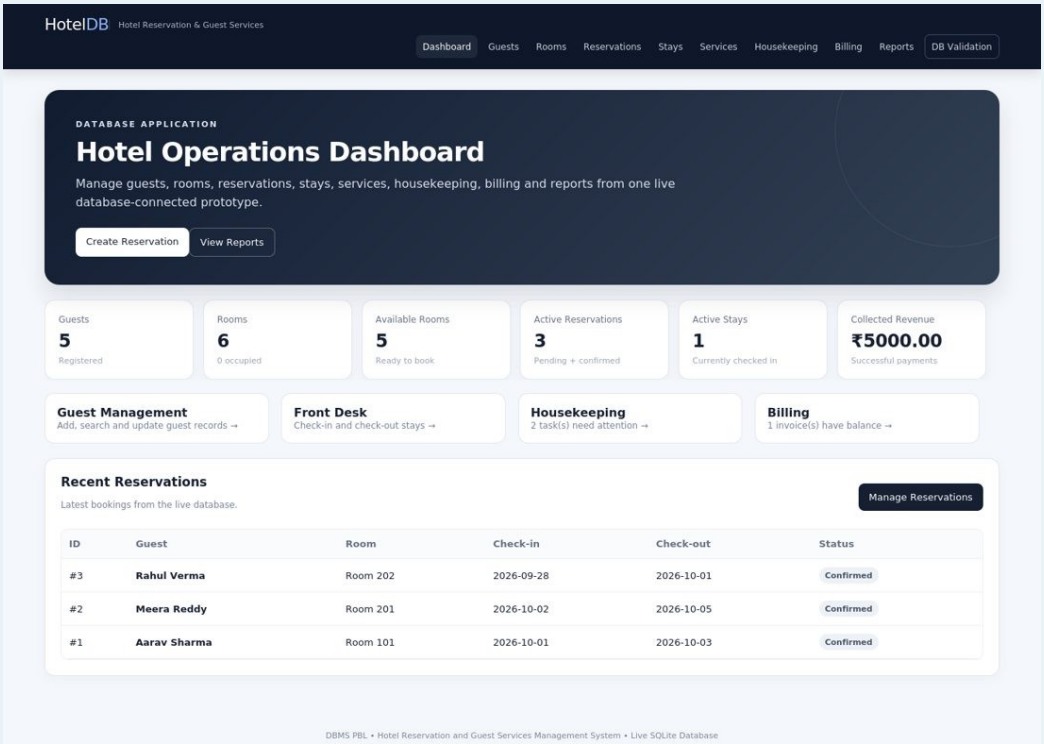



*Figure 1. Hotel Operations Dashboard reading live statistics from the implemented database*

## 3. Functional Dependencies

Functional dependencies (FDs) were identified for every relation. Because every relation uses a single-attribute surrogate primary key, each key determines all other attributes of its relation.

| Relation | Functional dependency |
|---|---|
| ROOM_TYPE | room_type_id → type_name, description, capacity, base_rate |
| ROOM | room_id → room_number, room_type_id, floor, status |
| RATE_PLAN | rate_plan_id → plan_name, description, rate, cancellation_policy |
| GUEST | guest_id → first_name, last_name, phone, email, address, id_proof |
| RESERVATION | reservation_id → guest_id, room_id, rate_plan_id, reservation_date, check_in_date, check_out_date, number_of_guests, status |
| STAY | stay_id → reservation_id, actual_check_in, actual_check_out, status |
| SERVICE_REQUEST | service_request_id → stay_id, service_type, request_date, status |
| HOUSEKEEPING_TASK | task_id → room_id, task_date, task_type, status, assigned_to |
| SERVICE_CHARGE | charge_id → stay_id, service_request_id, amount, charge_date |
| INVOICE | invoice_id → stay_id, invoice_date, room_charge, service_charge, tax, total_amount, status |
| PAYMENT | payment_id → invoice_id, payment_date, amount, payment_method, status |

**Candidate keys beyond the primary key** (each is enforced with a UNIQUE constraint): room_number, type_name, plan_name, guest.email, stay.reservation_id, invoice.stay_id.

# 4. Normalization

## 4.1 Starting point: one un-normalized booking sheet

A manual hotel register stores everything about a booking in one row. This shows exactly why Review 1 separated the data into 11 entities.

| res_id | guest_name | guest_phone | room_no | room_type | capacity | base_rate | services (repeating) |
|---|---|---|---|---|---|---|---|
| 1 | Aarav Sharma | 9876543210 | 101 | Standard | 2 | 2500 | Room Service 650; Laundry 300 |
| 2 | Aarav Sharma | 9876543210 | 102 | Standard | 2 | 2500 | Spa 1200 |
| 3 | Meera Reddy | 9876543211 | 201 | Deluxe | 3 | 4000 | (none) |

| Anomaly | What goes wrong in the single table |
|---|---|
| Insertion | A new room cannot be recorded until a guest books it; a new guest cannot be stored without a room. |
| Update | Changing the Standard room rate means editing every row of that type; changing a phone number must be repeated for each booking. |
| Deletion | Cancelling the only booking of room 201 also deletes the fact that Meera Reddy and the Deluxe type exist. |
| Repeating group | The services column holds several values in one cell, so charges cannot be summed or queried. |

## 4.2 First Normal Form (1NF)

- Every column holds a single atomic value. The repeating services column became rows in **service_request** and **service_charge**.
- Guest name is split into first_name and last_name.
- Every row is uniquely identified by a primary key (AUTOINCREMENT integer).

## 4.3 Second Normal Form (2NF)

- A table with a composite key (such as reservation + room) would allow room details to depend on part of the key.
- The design avoids this: every relation has a **single-attribute primary key**, so no non-key attribute can depend on only part of a key.
- Room details moved to **room** and **room_type**; guest details to **guest**; rate details to **rate_plan**.

## 4.4 Third Normal Form (3NF)

No non-key attribute may depend on another non-key attribute (no transitive dependency). Examples from this design:

| Transitive dependency removed | Resolved by |
|---|---|
| room_id → room_type_id → capacity, base_rate | capacity and base_rate stored once in ROOM_TYPE, referenced by room.room_type_id |
| reservation_id → guest_id → phone, email, address | Guest contact data stored once in GUEST, referenced by reservation.guest_id |

| Transitive dependency removed | Resolved by |
|---|---|
| reservation_id → rate_plan_id → rate, cancellation_policy | Pricing and policy stored once in RATE_PLAN |
| invoice_id → payment details | Payments stored in their own PAYMENT table (one invoice, many payments) |
| stay_id → service details | Service details split into SERVICE_REQUEST and SERVICE_CHARGE |

## 4.5 Normal form verification

| Relation | Primary key | Other candidate keys | 1NF | 2NF | 3NF |
|---|---|---|---|---|---|
| room_type | room_type_id | type_name | Yes | Yes | Yes |
| room | room_id | room_number | Yes | Yes | Yes |
| rate_plan | rate_plan_id | plan_name | Yes | Yes | Yes |
| guest | guest_id | email | Yes | Yes | Yes |
| reservation | reservation_id | none | Yes | Yes | Yes |
| stay | stay_id | reservation_id | Yes | Yes | Yes |
| service_request | service_request_id | none | Yes | Yes | Yes |
| housekeeping_task | task_id | none | Yes | Yes | Yes |
| service_charge | charge_id | service_request_id | Yes | Yes | Yes* |
| invoice | invoice_id | stay_id | Yes | Yes | Yes* |
| payment | payment_id | none | Yes | Yes | Yes |

### * Deliberate design decisions (honest note on 3NF)

**invoice** stores room_charge, service_charge, tax and total_amount. These can be calculated from other tables, but they are kept as a **billing snapshot**: an issued invoice must not change if a rate plan or tax rate is edited later. The application calculates them once, at checkout, in calculate_invoice().

**service_charge.stay_id** can also be reached through service_request.stay_id. It is kept so that charges can be summed per stay without an extra join, and because a charge may later exist without a request.

## 5. Logical Schema (3NF)

The final relational schema contains the same 11 entities as Review 1 with all keys and constraints made explicit. Legend: PK = primary key, FK = foreign key, UK = unique key.

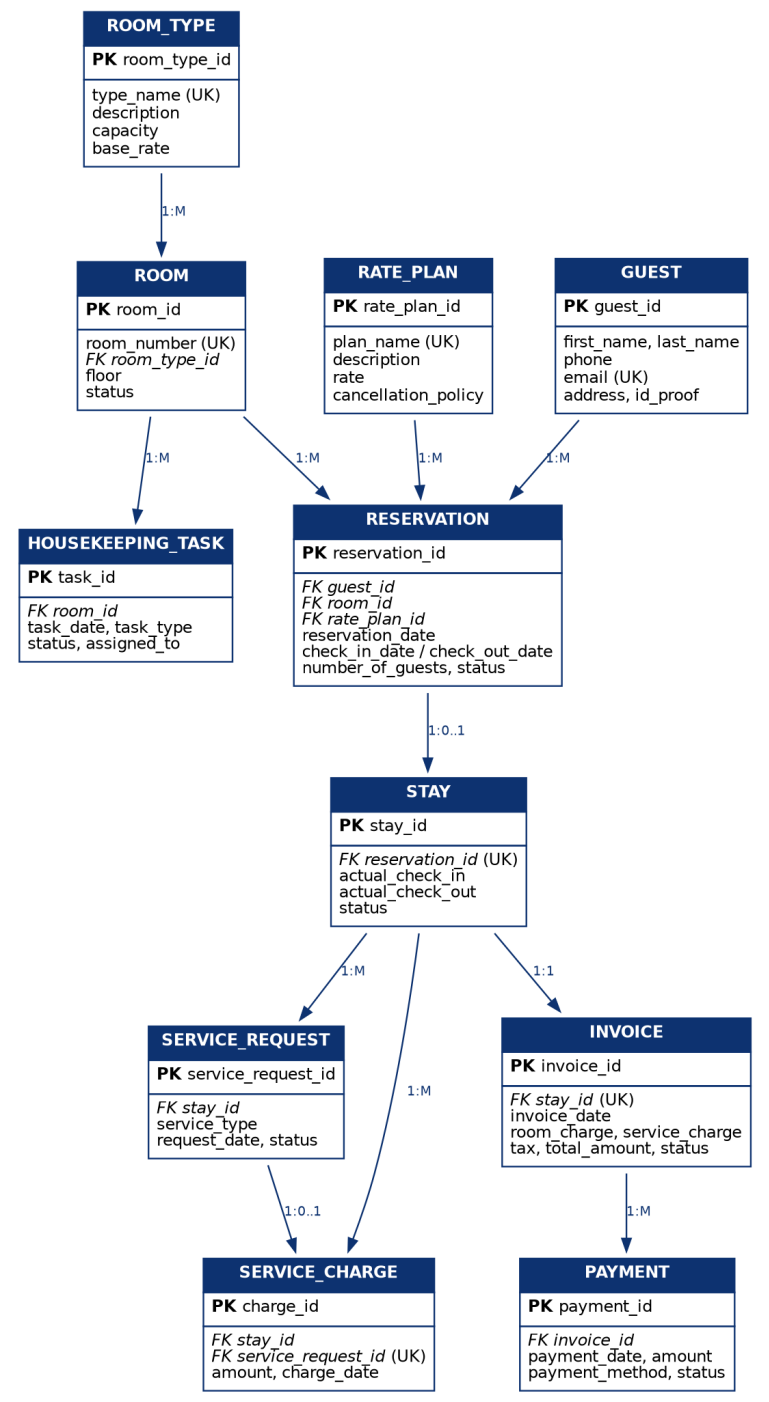



*Figure 2. Logical schema with keys and cardinalities (generated from the implemented schema.sql)*

# 6. Database Implementation (DDL)

The schema is created by **database/schema.sql** using CREATE TABLE IF NOT EXISTS, so it can be run repeatedly without errors. Representative statements are shown below.

## 6.1 Master data: ROOM_TYPE and ROOM

```
CREATE TABLE room_type (
  room_type_id INTEGER PRIMARY KEY AUTOINCREMENT,
  type_name    TEXT NOT NULL UNIQUE,
  description  TEXT,
  capacity     INTEGER NOT NULL CHECK (capacity > 0),
  base_rate    REAL NOT NULL CHECK (base_rate >= 0));

CREATE TABLE room (
  room_id INTEGER PRIMARY KEY AUTOINCREMENT,
  room_number TEXT NOT NULL UNIQUE,
  room_type_id INTEGER NOT NULL,
  floor INTEGER NOT NULL CHECK (floor >= 0),
  status TEXT NOT NULL DEFAULT 'Available'
    CHECK (status IN ('Available','Occupied','Maintenance','Cleaning')),
  FOREIGN KEY (room_type_id) REFERENCES room_type(room_type_id));
```

## 6.2 Transaction data: RESERVATION

```
CREATE TABLE reservation (
  reservation_id INTEGER PRIMARY KEY AUTOINCREMENT,
  guest_id INTEGER NOT NULL, room_id INTEGER NOT NULL,
  rate_plan_id INTEGER NOT NULL,
  reservation_date TEXT NOT NULL DEFAULT CURRENT_DATE,
  check_in_date TEXT NOT NULL, check_out_date TEXT NOT NULL,
  number_of_guests INTEGER NOT NULL CHECK (number_of_guests > 0),
  status TEXT NOT NULL DEFAULT 'Confirmed'
    CHECK (status IN ('Pending','Confirmed','Cancelled','Completed')),
  FOREIGN KEY (guest_id) REFERENCES guest(guest_id),
  FOREIGN KEY (room_id) REFERENCES room(room_id),
  FOREIGN KEY (rate_plan_id) REFERENCES rate_plan(rate_plan_id),
  CHECK (date(check_out_date) > date(check_in_date)));
```

## 6.3 Billing: INVOICE and PAYMENT, plus indexes

```
CREATE TABLE invoice (
  invoice_id INTEGER PRIMARY KEY AUTOINCREMENT,
  stay_id INTEGER NOT NULL UNIQUE,            -- one invoice per stay
  invoice_date TEXT NOT NULL DEFAULT CURRENT_DATE,
  room_charge REAL NOT NULL DEFAULT 0 CHECK (room_charge >= 0),
  service_charge REAL NOT NULL DEFAULT 0 CHECK (service_charge >= 0),
  tax REAL NOT NULL DEFAULT 0 CHECK (tax >= 0),
  total_amount REAL NOT NULL DEFAULT 0 CHECK (total_amount >= 0),
  status TEXT NOT NULL DEFAULT 'Unpaid'
    CHECK (status IN ('Unpaid','Partially Paid','Paid')),
  FOREIGN KEY (stay_id) REFERENCES stay(stay_id));

CREATE INDEX idx_reservation_dates ON reservation(room_id, check_in_date, check_out_date);
CREATE INDEX idx_service_stay ON service_request(stay_id);
CREATE INDEX idx_payment_invoice ON payment(invoice_id);
CREATE UNIQUE INDEX idx_service_charge_request_unique
  ON service_charge(service_request_id) WHERE service_request_id IS NOT NULL;
```

### 6.4 Constraint summary

| Constraint type | Where it is used | Purpose |
|---|---|---|
| PRIMARY KEY | All 11 tables (AUTOINCREMENT) | Unique identity of every row |
| FOREIGN KEY | 11 relationships (e.g. reservation → guest, room, rate_plan) | Referential integrity; no orphan records |
| UNIQUE | room_number, type_name, plan_name, guest.email, stay.reservation_id, invoice.stay_id | Prevent duplicates; enforce 1:1 and 1:0..1 relationships |
| CHECK | capacity > 0, rates >= 0, guests > 0, check-out > check-in, payment amount > 0, status lists, payment method list | Valid values only |
| DEFAULT | status columns, reservation_date, invoice_date, payment_date, request_date | Consistent automatic values |
| INDEX | reservation(room_id, dates), service_request(stay_id), payment(invoice_id) | Faster overlap checks, service lookup and balance calculation |

## 7. Key Rules: How Each Is Enforced

Review 1 listed six key rules. Each one is now enforced, some by the database itself and some by the application before the SQL is executed.

| Rule (Review 1) | Database level | Application level (Flask) |
|---|---|---|
| 01 No overlapping stays | idx_reservation_dates supports the lookup | reservation_conflict() rejects an active booking whose dates overlap the same room |
| 02 Valid dates | CHECK (check_out_date > check_in_date) | Rejects past check-in dates and invalid ranges with a message |
| 03 Room capacity | capacity > 0 CHECK on room_type | Compares number_of_guests with the room type capacity |
| 04 Valid charges | CHECK (amount >= 0); unique charge per request | Rejects charge <= 0 and duplicate charge for one request |
| 05 Payment limit | CHECK (amount > 0) | Rejects a payment larger than the outstanding balance |
| 06 Referential integrity | FOREIGN KEYs with foreign_keys = ON | Guests with reservation history cannot be deleted |

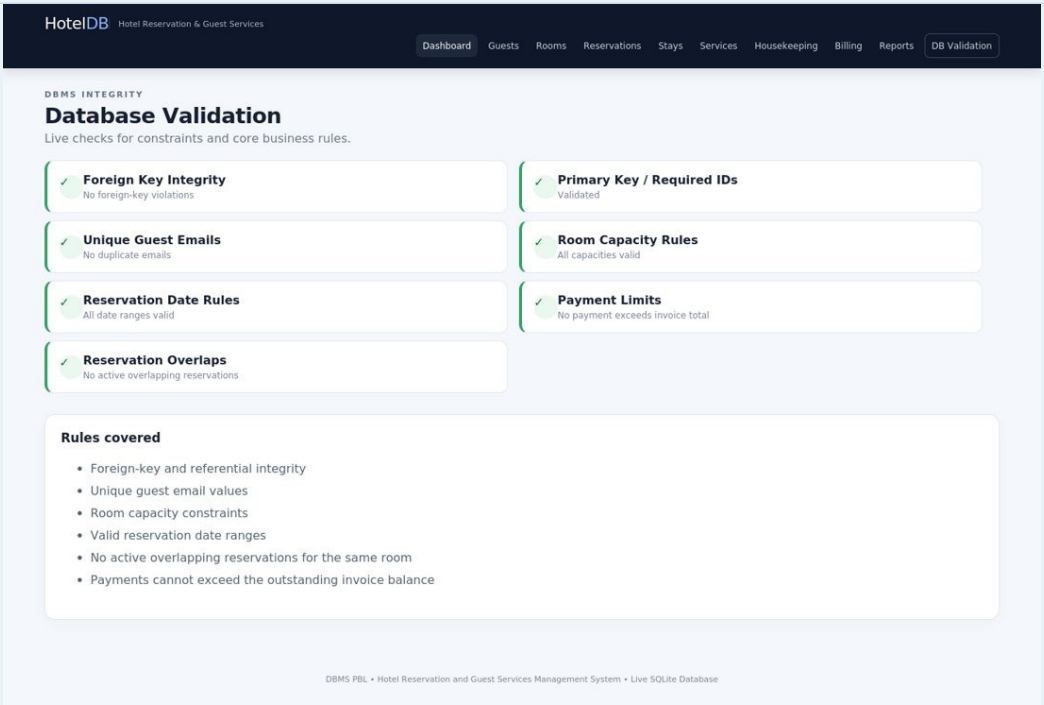



*Figure 3. Database Validation screen: live integrity checks over the implemented database*

## 8. Sample Data

database/seed.sql inserts realistic data using INSERT OR IGNORE, and only when the guest table is empty, so the prototype opens with meaningful records for demonstration.

| Table | Rows | Sample content |
|---|---|---|
| room_type | 3 | Standard (cap. 2, Rs 2,500), Deluxe (cap. 3, Rs 4,000), Suite (cap. 4, Rs 6,500) |
| rate_plan | 3 | Standard Flexible, Deluxe Flexible, Suite Premium with cancellation policies |
| room | 6 | 101, 102 (Standard); 201, 202 (Deluxe); 301, 302 (Suite, 302 under Maintenance) |
| guest | 5 | Aarav Sharma, Meera Reddy, Rahul Verma, Ananya Patel, Vikram Singh |
| reservation | 3 | Two future confirmed bookings and one current booking |
| stay | 1 | Active stay created from a reservation |
| service_request / service_charge | 2 / 2 | Room Service Rs 650 and Laundry Rs 300 |
| housekeeping_task | 3 | Deep Cleaning (Pending), Room Cleaning (In Progress), Checkout Cleaning (Completed) |
| invoice / payment | 1 / 1 | Partially paid invoice with one UPI payment of Rs 5,000 |

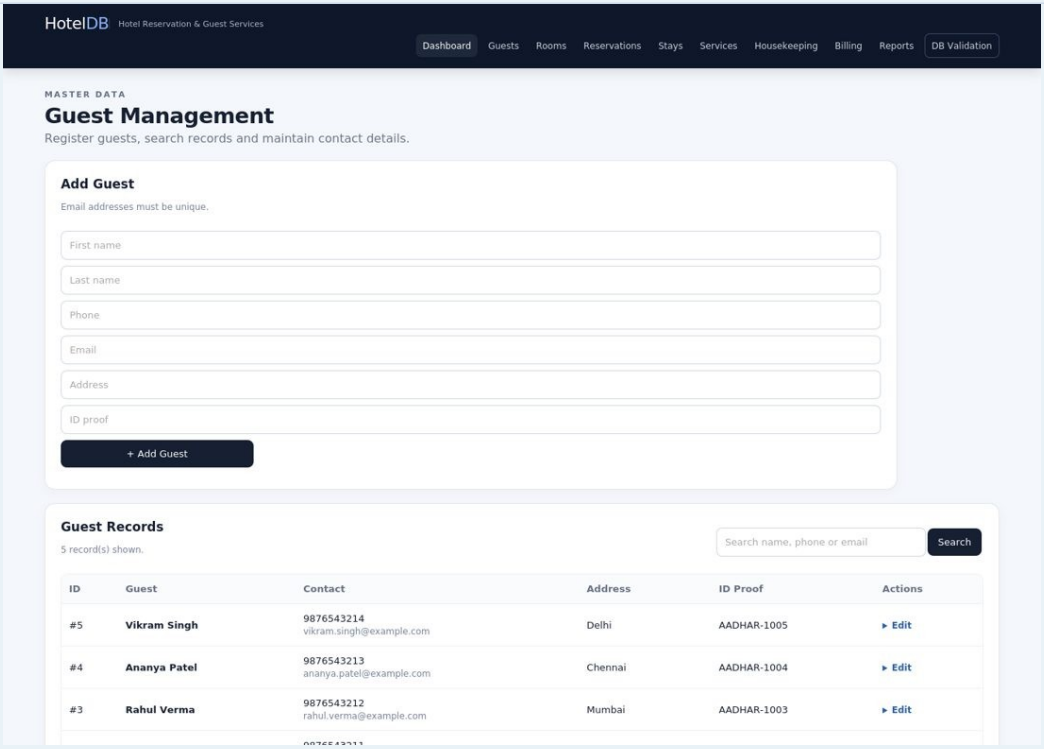



*Figure 4. Guest Management screen showing seeded guest records*

## 9. Conclusion and Next Phase

The Review 1 conceptual design has been normalized to 3NF, implemented as an 11-table SQLite database with primary keys, foreign keys, unique and check constraints, indexes and sample data, and connected to a working web prototype. Anomalies identified in the un-normalized register (insertion, update, deletion and repeating groups) are eliminated by the decomposition.

> **NEXT PHASE (Review 3)**
>
> SQL Queries and Reports → Complete application workflow → Testing (21 test cases) → Results, limitations and future scope